

Reflection Log

Noorinder

Credit 2

Assignment Name: MySavings

How has your program changed from planning to coding to now?

At first, the program was planned to create a simple PiggyBank application. Users could add pennies, nickels, dimes, and quarters to the PiggyBank, view their total savings, and withdraw money. The plan also included saving the PiggyBank's state to a file and loading it back when the program restarted. This way, the user's savings were always stored safely between sessions.

During coding, I faced challenges that required me to make changes to improve the program's reliability and functionality:

1. File Not Found Issues

- **Problem:** When the program tried to load the PiggyBank data from a file, it sometimes couldn't find the file. This caused the program to throw a `FileNotFoundException` and crash, especially when the program was run for the first time and the file didn't exist yet.
- **Solution:** I added code to check if the file was missing. If it was, the program would create a new PiggyBank object instead of crashing. This ensured the program always worked, even if no file was present.

Before Code:

```
FileInputStream fileIn = new FileInputStream("../Chapter11/src/Mastery/piggybank.dat");
ObjectInputStream readBank = new ObjectInputStream(fileIn);
piggyBank = (PiggyBank) readBank.readObject(); // Crashes if the file is missing
readBank.close();
```

After Fix:

```

try {
    FileInputStream fileIn = new FileInputStream("../Chapter11/src/Mastery/piggybank.dat");
    ObjectInputStream readBank = new ObjectInputStream(fileIn);
    piggyBank = (PiggyBank) readBank.readObject();
    readBank.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found. Starting with a new PiggyBank.");
    piggyBank = new PiggyBank(); // Initializes a new PiggyBank if no file exists
}

```

2. Data Corruption During Save

- **Problem:** Sometimes, the file used to save the PiggyBank data became corrupted. This happened when the program didn't properly close the file after writing to it. If the program crashed or was stopped while saving, the file would remain partially written, making it unreadable.
- **Solution:** I used a `try-with-resources` block to ensure the file was always closed, even if an error occurred. This reduced the chances of corruption.

Before Code:

```

ObjectOutputStream writeBank = new ObjectOutputStream(fileOut);
writeBank.writeObject(piggyBank); // Doesn't close the file if an exception occurs
writeBank.close();

```

After Fix:

```

try (ObjectOutputStream writeBank = new ObjectOutputStream(fileOut)) {
    writeBank.writeObject(piggyBank); // Automatically closes the file after writing
} catch (IOException e) {
    System.err.println("Error saving PiggyBank data: " + e.getMessage());
}

```

3. Invalid User Input

- **Problem:** If the user entered something other than a number (e.g., a letter or symbol), the program would throw an error and crash. This happened because the program didn't check if the input was valid before trying to use it.

- **Solution:** I added input validation to ensure the user entered numbers only. If the input was invalid, the program showed an error message and asked the user to try again.

Before Code:

```
choice = scanner.nextInt(); // Assumes the input is always a valid number
```

After Fix:

```
try {  
    System.out.print("Enter your choice: ");  
    choice = Integer.parseInt(scanner.nextLine()); // Safely converts input to an integer  
} catch (NumberFormatException e) {  
    System.out.println("Invalid input. Please enter a number.");  
    choice = -1; // Sets an invalid choice to loop back to the menu  
}  
  
}
```

4. Serialization Compatibility

- **Problem:** After making changes to the `PiggyBank` class, older files saved with the previous version of the class couldn't be loaded. This caused a `ClassNotFoundException` and made it impossible to restore the saved data.
- **Solution:** I added a `serialVersionUID` to the `PiggyBank` class. This ensured that files saved with different versions of the class could still be loaded correctly.

Code Example:

```
// Added in PiggyBank class  
private static final long serialVersionUID = 1L; // Ensures compatibility between versions
```